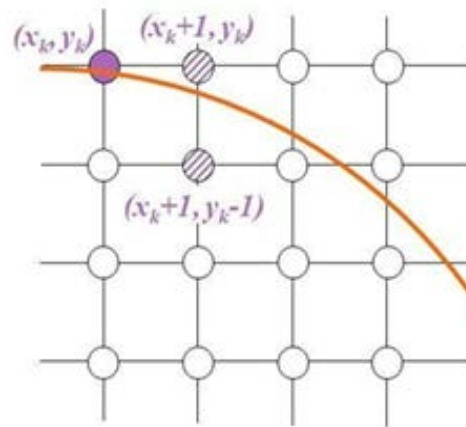
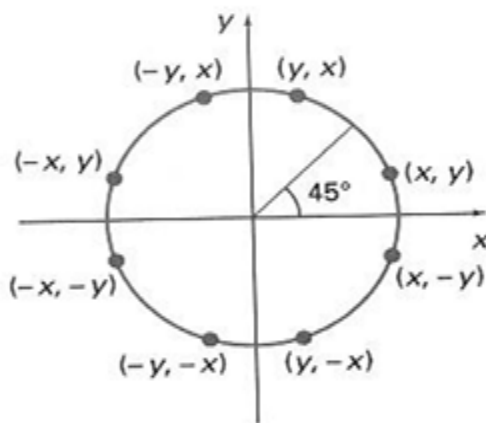


TITLE: IMPLEMENTATION OF MIDPOINT CIRCLE GENERATION ALGORITHM

INTRODUCTION

The Midpoint Circle Algorithm, also known as Bresenham's Circle Algorithm, is a computer graphics method used to draw a circle efficiently using only integer arithmetic. The algorithm uses the symmetry of a circle to reduce the number of calculations needed. Instead of computing all 360° of the circle, it calculates only points for the first 45° (first octant), and by symmetry, the corresponding points in all eight octants are plotted.



This makes the algorithm fast and suitable for raster graphics applications where pixel activation must be computed precisely.

ALGORITHM

Step 1: Input the radius r and center of circle (x_c, y_c) .

Set the starting point on the circle at $(x, y) = (0, r)$.

Step 2: Calculate the initial decision parameter:

$$p_0 = 1 - r$$

Step 3: Set $k = 0$.

For each point (x_k, y_k) , perform the following test:

- If $p_k < 0$, then the next point on the circle is: (x_{k+1}, y_k)

b. Update the decision parameter:

$$p_{k+1} = p_k + 2x_k + 1$$

b. Else ($p_k \geq 0$), the next point on the circle is:

$$(x_{k+1}, y_k - 1)$$

Update the decision parameter:

$$p_{k+1} = p_k + 2x_k + 1 - 2y_k$$

Step 4: For every calculated point, compute and plot its symmetric points in all 8 octants.

Step 5: Move each calculated coordinate into the actual circle location by adding center values:

$$x = x + x_c$$

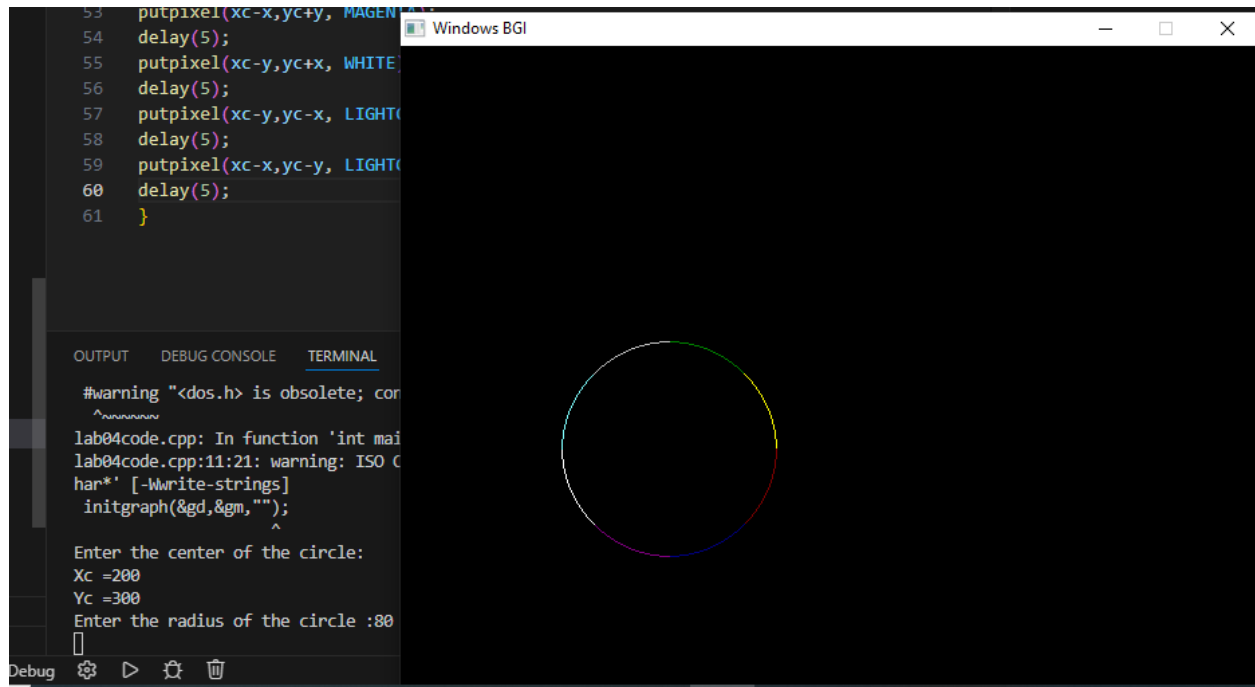
$$y = y + y_c$$

Step 6: Repeat Steps 3–5 until $x \geq y$ (i.e., until you complete drawing the circle portion)

Symmetric pixel positions:

- `putpixel(xc+x,yc-y,GREEN); //For pixel (x,y)`
- `putpixel(xc+y,yc-x, GREEN); //For pixel (y,x)`
- `putpixel(xc+y,yc+x, GREEN); //For pixel (y,-x)`
- `putpixel(xc+x,yc+y, GREEN); //For pixel (x,-y)`
- `putpixel(xc-x,yc+y, GREEN); //For pixel (-x,-y)`
- `putpixel(xc-y,yc+x, GREEN); //For pixel (-y,-x)`
- `putpixel(xc-y,yc-x, GREEN); //For pixel (-y,x)`
- `putpixel(xc-x,yc-y, GREEN); //For pixel (-x,y)`

OUTPUT



The screenshot displays a C++ IDE with a code editor on the left and a terminal window on the right. The code editor shows the implementation of the Midpoint Circle Algorithm, which uses a loop to calculate the pixels of a circle by leveraging symmetry. The terminal window shows the program's execution, including user input for the center coordinates (Xc=200, Yc=300) and the radius (80). The output is a window titled 'Windows BGI' showing a black background with a circle drawn in a rainbow color gradient.

```
53 putpixel(xc-x,yc+y, MAGENTA);
54 delay(5);
55 putpixel(xc-y,yc+x, WHITE);
56 delay(5);
57 putpixel(xc-y,yc-x, LIGHTBLUE);
58 delay(5);
59 putpixel(xc-x,yc-y, LIGHTGREEN);
60 delay(5);
61 }
```

OUTPUT DEBUG CONSOLE TERMINAL

```
#warning "<dos.h> is obsolete; consider using <windows.h> instead"
lab04code.cpp: In function 'int main()':
lab04code.cpp:11:21: warning: ISO C++11 requires explicit narrowing conversions [-Wwrite-strings]
    initgraph(&gd,&gm,"");
    ^
Enter the center of the circle:
Xc =200
Yc =300
Enter the radius of the circle :80

```

CONCLUSION

The Midpoint Circle Algorithm allows circles to be drawn efficiently in computer graphics using simple integer calculations. By utilizing circle symmetry and plotting only one octant, the algorithm reduces computation while producing accurate pixel activation for a complete circular shape. This makes it a foundational technique in computer graphics for rendering curved shapes.